

Statistical Foundations of Prior-Data Fitted Networks [Nagler '23]

Salih Bora Öztürk & Gurmeher Singh Puri

19 June 2026

University of Freiburg

Seminar on Foundation Models for Structured Data - SS26

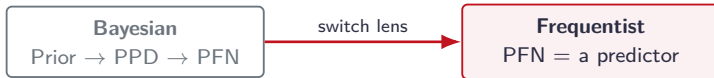
Group 4



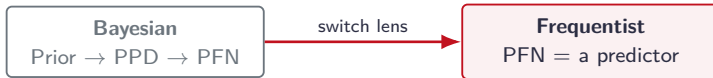
PFNs approximate PPD

PFNs approximate PPD

Why a frequentist view of PFNs?



Why a frequentist view of PFNs?



The puzzle



Bayes is silent past 1000.

Why a frequentist view of PFNs?

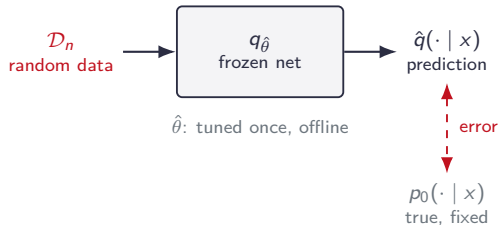


The puzzle



Bayes is silent past 1000.

The reframe: a PFN is just a predictor

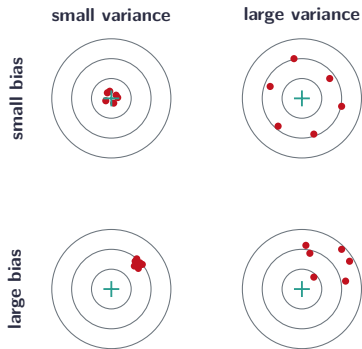


$$\text{error} = \text{bias} + \text{variance}$$

wrong on average wobble from random $\mathcal{D}_n$

Bias and variance

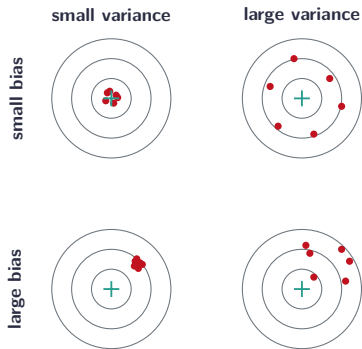
Each **dart** = a prediction from one random dataset; the **bullseye** = the truth p_0 .



Bias and variance

Each **dart** = a prediction from one random dataset; the **bullseye** = the truth p_0 .

Variance = how **scattered** the darts are.



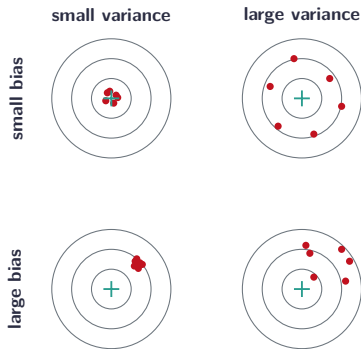
Bias and variance

Each **dart** = a prediction from one random dataset; the **bullseye** = the truth p_0 .

Variance = how **scattered** the darts are.

Bias = how far the darts' **centre** is from the bullseye.

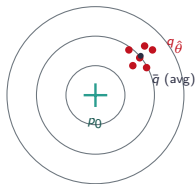
Want: small bias *and* small variance (top-left).



Bias and variance: the decomposition

$$\underbrace{q_{\hat{\theta}}(y \mid x, \mathcal{D}_n) - p_0(y \mid x)}_{\text{error}}$$
$$= \underbrace{q_{\hat{\theta}}(y \mid x, \mathcal{D}_n) - \mathbb{E}_{\mathcal{D}_n \sim p_0^n}[q_{\hat{\theta}}(y \mid x, \mathcal{D}_n)]}_{\text{variance}}$$
$$+ \underbrace{\mathbb{E}_{\mathcal{D}_n \sim p_0^n}[q_{\hat{\theta}}(y \mid x, \mathcal{D}_n)] - p_0(y \mid x)}_{\text{bias}}$$

$\bar{q} = \mathbb{E}_{\mathcal{D}_n}[q_{\hat{\theta}}]$ — the average prediction.

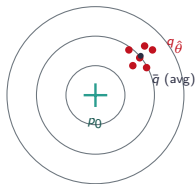


Bias and variance: the decomposition

$$\underbrace{q_{\hat{\theta}}(y \mid x, \mathcal{D}_n) - p_0(y \mid x)}_{\text{error}}$$
$$= \underbrace{q_{\hat{\theta}}(y \mid x, \mathcal{D}_n) - \mathbb{E}_{\mathcal{D}_n \sim p_0^n}[q_{\hat{\theta}}(y \mid x, \mathcal{D}_n)]}_{\text{variance}}$$
$$+ \underbrace{\mathbb{E}_{\mathcal{D}_n \sim p_0^n}[q_{\hat{\theta}}(y \mid x, \mathcal{D}_n)] - p_0(y \mid x)}_{\text{bias}}$$

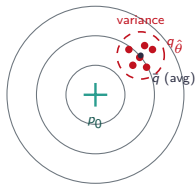
$\bar{q} = \mathbb{E}_{\mathcal{D}_n}[q_{\hat{\theta}}]$ — the average prediction.

error: $q_{\hat{\theta}}$ = our prediction, p_0 = the truth.



Bias and variance: the decomposition

$$\underbrace{q_{\hat{\theta}}(y \mid x, \mathcal{D}_n) - p_0(y \mid x)}_{\text{error}} \\ = \underbrace{q_{\hat{\theta}}(y \mid x, \mathcal{D}_n) - \mathbb{E}_{\mathcal{D}_n \sim p_0^n}[q_{\hat{\theta}}(y \mid x, \mathcal{D}_n)]}_{\text{variance}} \\ + \underbrace{\mathbb{E}_{\mathcal{D}_n \sim p_0^n}[q_{\hat{\theta}}(y \mid x, \mathcal{D}_n)] - p_0(y \mid x)}_{\text{bias}}$$



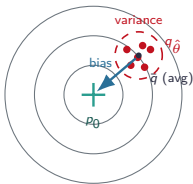
$\bar{q} = \mathbb{E}_{\mathcal{D}_n}[q_{\hat{\theta}}]$ — the average prediction.

error: $q_{\hat{\theta}}$ = our prediction, p_0 = the truth.

variance: prediction $q_{\hat{\theta}}$ vs its average $\bar{q}$.

Bias and variance: the decomposition

$$\begin{aligned} & \underbrace{q_{\hat{\theta}}(y \mid x, \mathcal{D}_n) - p_0(y \mid x)}_{\text{error}} \\ &= \underbrace{q_{\hat{\theta}}(y \mid x, \mathcal{D}_n) - \mathbb{E}_{\mathcal{D}_n \sim p_0^n}[q_{\hat{\theta}}(y \mid x, \mathcal{D}_n)]}_{\text{variance}} \\ &+ \underbrace{\mathbb{E}_{\mathcal{D}_n \sim p_0^n}[q_{\hat{\theta}}(y \mid x, \mathcal{D}_n)] - p_0(y \mid x)}_{\text{bias}} \end{aligned}$$



$\bar{q} = \mathbb{E}_{\mathcal{D}_n}[q_{\hat{\theta}}]$ — the average prediction.

error: $q_{\hat{\theta}}$ = our prediction, p_0 = the truth.

variance: prediction $q_{\hat{\theta}}$ vs its average $\bar{q}$.

bias: average $\bar{q}$ vs the truth p_0 .

How can we reduce bias and variance?

We can reduce each part of the error with its **own tool**:

How can we reduce bias and variance?

We can reduce each part of the error with its **own tool**:

Variance the scatter

- Symmetry
- Diminishing sensitivity

How can we reduce bias and variance?

We can reduce each part of the error with its **own tool**:

Variance the scatter

- Symmetry
- Diminishing sensitivity

Bias the offset

- Locality

Symmetry

A predictor is **symmetric** if shuffling the training rows never changes its answer
(rows are iid $\Rightarrow$ their order is meaningless).

Take two predictors: $f = \text{not symmetric}$ vs $\tilde{f} = \text{a symmetric one}$.

Symmetry

A predictor is **symmetric** if shuffling the training rows never changes its answer
(rows are iid $\Rightarrow$ their order is meaningless).

Take two predictors: $f = \text{not symmetric}$ vs $\tilde{f} = \text{a symmetric one}$.

$$\mathbb{E}[\tilde{f}] = \mathbb{E}[f] \quad \text{same expected value} \Rightarrow \text{same bias}$$

Symmetry

A predictor is **symmetric** if shuffling the training rows never changes its answer
(rows are iid $\Rightarrow$ their order is meaningless).

Take two predictors: $f = \text{not symmetric}$ vs $\tilde{f} = \text{a symmetric one}$.

$$\mathbb{E}[\tilde{f}] = \mathbb{E}[f] \quad \text{same expected value} \Rightarrow \text{same bias}$$

$$\text{Var}[\tilde{f}] \leq \text{Var}[f] \quad \text{the symmetric one has lower variance}$$

Diminishing sensitivity

Change *one* of the n data points — how much does the prediction move?

attention weights sum to 1, so each sample gets weight $\approx 1/n$ — more rows, the weight spreads out:

few samples



flip one $\Rightarrow$ big swing ($\approx \frac{1}{4}$)

many samples



flip one $\Rightarrow$ tiny swing ($\approx \frac{1}{n}$)

Diminishing sensitivity

Change *one* of the n data points — how much does the prediction move?

attention weights sum to 1, so each sample gets weight $\approx 1/n$ — more rows, the weight spreads out:

few samples



flip one $\Rightarrow$ big swing ($\approx \frac{1}{4}$)

many samples



flip one $\Rightarrow$ tiny swing ($\approx \frac{1}{n}$)

Swap one sample $\mathcal{D}_n \rightarrow \mathcal{D}'_n$ — the prediction moves by *at most*:

$$\left| q_{\hat{\theta}}(\mathcal{D}_n) - q_{\hat{\theta}}(\mathcal{D}'_n) \right| \leq L n^{-\alpha}$$

Diminishing sensitivity

Change *one* of the n data points — how much does the prediction move?

attention weights sum to 1, so each sample gets weight $\approx 1/n$ — more rows, the weight spreads out:

few samples



flip one $\Rightarrow$ big swing ($\approx \frac{1}{4}$)

many samples



flip one $\Rightarrow$ tiny swing ($\approx \frac{1}{n}$)

Swap one sample $\mathcal{D}_n \rightarrow \mathcal{D}'_n$ — the prediction moves by *at most*:

$$|q_{\hat{\theta}}(\mathcal{D}_n) - q_{\hat{\theta}}(\mathcal{D}'_n)| \leq L n^{-\alpha}$$

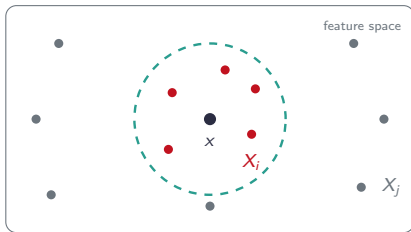
$\mathcal{D}'_n$: $\mathcal{D}_n$ with **one sample swapped** L : a fixed constant $n^{-\alpha}$: **shrinks as n grows** (α = how fast)

$\Rightarrow$ more rows $\Rightarrow$ each single sample's influence **diminishes**.

Locality: only nearby points should count

Variance handled — now the **bias**. The tool: **locality**.

To predict at the query x , which training points should matter?

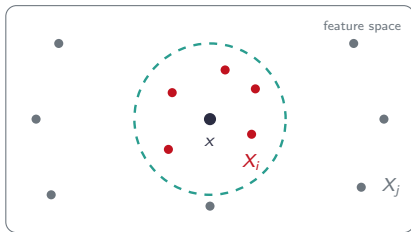


- x (the query)
- sample X_i near x
- sample X_j far from x (also a sample)

Locality: only nearby points should count

Variance handled — now the **bias**. The tool: **locality**.

To predict at the query x , which training points should matter?



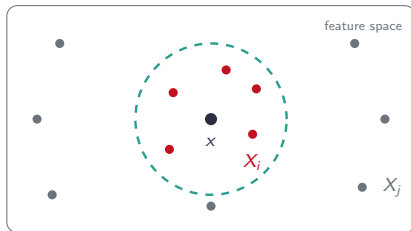
● x (the query) ● sample X_i near x ● sample X_j far from x (also a sample)

Inside the ring (**near** x , red) drives the answer; the far (grey) points are ignored.

Locality: only nearby points should count

Variance handled — now the **bias**. The tool: **locality**.

To predict at the query x , which training points should matter?



● x (the query) ● sample X_i near x ● sample X_j far from x (also a sample)

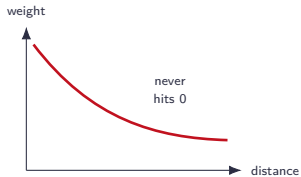
Inside the ring (**near** x , red) drives the answer; the far (grey) points are ignored.

[Thm 5.4] an *unbiased* predictor must be **local**: flip the far-away labels $\rightarrow$ the prediction doesn't change.

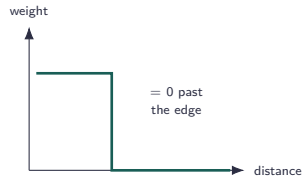
Transformers aren't local

Attention is a **soft** weighting: it favours samples similar to x , but never drops the far ones to zero.

attention (soft)

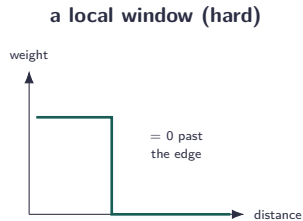


a local window (hard)



Transformers aren't local

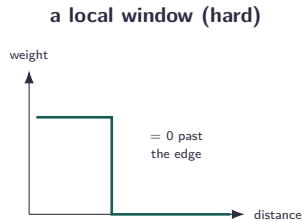
Attention is a **soft** weighting: it favours samples similar to x , but never drops the far ones to zero.



Far points **always leak in** $\Rightarrow$ you *can't* flip them freely $\Rightarrow$ **not local**.

Transformers aren't local

Attention is a **soft** weighting: it favours samples similar to x , but never drops the far ones to zero.

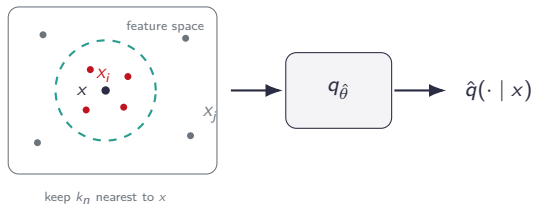


Far points **always leak in** $\Rightarrow$ you *can't* flip them freely $\Rightarrow$ **not local**.

$\Rightarrow$ the bias does *not* vanish — TabPFN's bias flattens out around $n \approx 1000$ [Fig. 1].

Localized PFNs

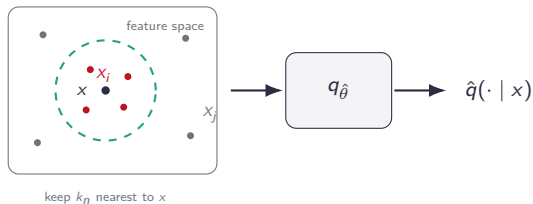
A simple add-on to *any* frozen PFN — localize at prediction time.



• x (query) • near samples X_i (kept) • far samples X_j (dropped)

Localized PFNs

A simple add-on to *any* frozen PFN — localize at prediction time.

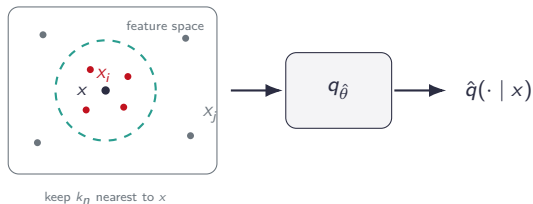


• x (query) • near samples X_i (kept) • far samples X_j (dropped)

Why it works: near x the truth is $\approx$ **flat** $\Rightarrow$ easy to fit $\Rightarrow$ **bias drops**. (cost: fewer points $\Rightarrow$ slightly more variance.)

Localized PFNs

A simple add-on to *any* frozen PFN — localize at prediction time.



• x (query) • near samples X_i (kept) • far samples X_j (dropped)

Why it works: near x the truth is $\approx$ **flat** $\Rightarrow$ easy to fit $\Rightarrow$ **bias drops**. (cost: fewer points $\Rightarrow$ slightly more variance.)

Does it actually break the bias floor? We'll put it to the test at the end [Fig. 1].

Two more concrete predictors, each with a different bias/variance behaviour:

1. **Window smoother**
2. **Classification trees**

Two more concrete predictors, each with a different bias/variance behaviour:

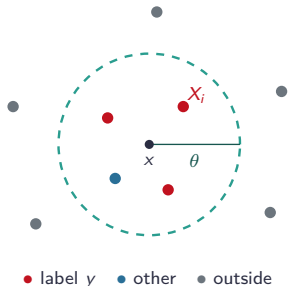
1. **Window smoother**
2. **Classification trees**

(the Localized PFN above is the third.)

Window smoother

$$q_{\theta}(y \mid x, \mathcal{D}_n) = \frac{\sum_i \mathbf{1}(Y_i = y) \mathbf{1}(\|X_i - x\| < \theta)}{\sum_i \mathbf{1}(\|X_i - x\| < \theta)}$$

dots = training points X_i (with label Y_i); x = the query.

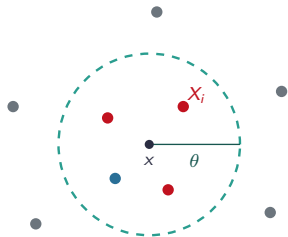


Window smoother

$$q_{\theta}(y \mid x, \mathcal{D}_n) = \frac{\sum_i \mathbf{1}(Y_i = y) \mathbf{1}(\|X_i - x\| < \theta)}{\sum_i \mathbf{1}(\|X_i - x\| < \theta)}$$

dots = training points X_i (with label Y_i); x = the query.

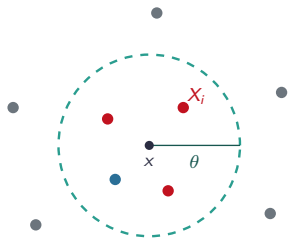
$\|X_i - x\| < \theta$: X_i is **inside the window**.



• label y • other • outside

Window smoother

$$q_{\theta}(y \mid x, \mathcal{D}_n) = \frac{\sum_i \mathbf{1}(Y_i = y) \mathbf{1}(\|X_i - x\| < \theta)}{\sum_i \mathbf{1}(\|X_i - x\| < \theta)}$$



● label y ● other ● outside

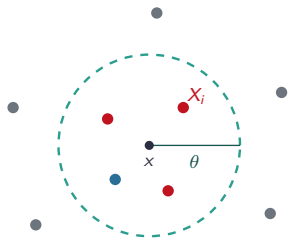
dots = training points X_i (with label Y_i); x = the query.

$\|X_i - x\| < \theta$: X_i is **inside the window**.

$\mathbf{1}(Y_i = y)$: its label is y (red).

Window smoother

$$q_{\theta}(y \mid x, \mathcal{D}_n) = \frac{\sum_i \mathbf{1}(Y_i = y) \mathbf{1}(\|X_i - x\| < \theta)}{\sum_i \mathbf{1}(\|X_i - x\| < \theta)}$$



● label y ● other ● outside

dots = training points X_i (with label Y_i); x = the query.

$\|X_i - x\| < \theta$: X_i is **inside the window**.

$\mathbf{1}(Y_i = y)$: its label is y (red).

$$q_{\theta} = \frac{\#\{\text{inside \& } y\}}{\#\{\text{inside}\}} = \frac{3}{4}.$$

Window smoother — shrink the window, kill the bias

Now let the radius *shrink with n*: replace θ by $a_n \theta$.

$$q_\theta(y \mid x, \mathcal{D}_n) = \frac{\sum_i \mathbf{1}(Y_i = y) \mathbf{1}(\|X_i - x\| < a_n \theta)}{\sum_i \mathbf{1}(\|X_i - x\| < a_n \theta)}.$$

Window smoother — shrink the window, kill the bias

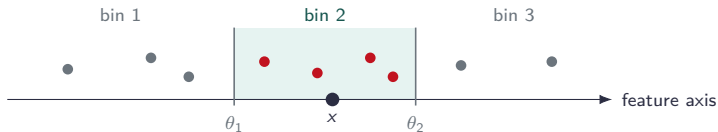
Now let the radius *shrink with n* : replace θ by $a_n \theta$.

$$q_\theta(y \mid x, \mathcal{D}_n) = \frac{\sum_i \mathbf{1}(Y_i = y) \mathbf{1}(\|X_i - x\| < a_n \theta)}{\sum_i \mathbf{1}(\|X_i - x\| < a_n \theta)}.$$

Optimal schedule $a_n = n^{-1/(4+d)}$ ($d = \text{\#features}$): **bias $\rightarrow 0$ as data grows.**

Classification trees

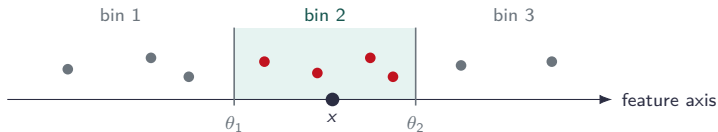
A tree **cuts feature space into bins**; to predict at x , report the **label-mix** of x 's bin.



x lands in **bin 2** $\Rightarrow$ use only the points there.

Classification trees

A tree **cuts feature space into bins**; to predict at x , report the **label-mix** of x 's bin.



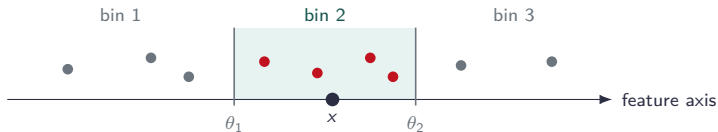
x lands in **bin 2** $\Rightarrow$ use only the points there.

$$q_{\theta}(y \mid x, \mathcal{D}_n) = \sum_j \mathbf{1}(x \in \text{bin}_j) \frac{\sum_i \mathbf{1}(Y_i = y) \mathbf{1}(X_i \in \text{bin}_j)}{\sum_i \mathbf{1}(X_i \in \text{bin}_j)}$$

which bin holds x

Classification trees

A tree **cuts feature space into bins**; to predict at x , report the **label-mix** of x 's bin.



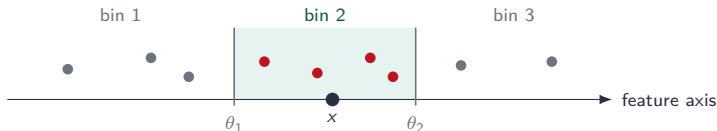
x lands in **bin 2** $\Rightarrow$ use only the points there.

$$q_{\theta}(y \mid x, \mathcal{D}_n) = \sum_j \mathbf{1}(x \in \text{bin}_j) \frac{\sum_i \mathbf{1}(Y_i = y) \mathbf{1}(X_i \in \text{bin}_j)}{\sum_i \mathbf{1}(X_i \in \text{bin}_j)}$$

which bin holds x the points in it

Classification trees

A tree **cuts feature space into bins**; to predict at x , report the **label-mix** of x 's bin.



x lands in **bin 2** $\Rightarrow$ use only the points there.

$$q_{\theta}(y \mid x, \mathcal{D}_n) = \sum_j \mathbf{1}(x \in \text{bin}_j) \frac{\sum_i \mathbf{1}(Y_i = y) \mathbf{1}(X_i \in \text{bin}_j)}{\sum_i \mathbf{1}(X_i \in \text{bin}_j)}$$

which bin holds x the points in it ... with label y $\Rightarrow$ the bin's **label fraction**

Classification trees: the bias is stuck

$$\text{bias} = \underbrace{\Pr_0(Y = y \mid X \in \text{bin})}_{\text{truth averaged over the bin}} - \underbrace{p_0(y \mid x)}_{\text{truth at } x}$$

Bins are **fixed** $\Rightarrow$ each bin blurs the same patch $\Rightarrow$ **bias can't shrink** (more data = tighter, not truer).

Classification trees: the bias is stuck

$$\text{bias} = \underbrace{\Pr_0(Y = y \mid X \in \text{bin})}_{\text{truth averaged over the bin}} - \underbrace{p_0(y \mid x)}_{\text{truth at } x}$$

Bins are **fixed** $\Rightarrow$ each bin blurs the same patch $\Rightarrow$ **bias can't shrink** (more data = tighter, not truer).

Fix: average **many trees** with *different* splits: $q = \sum_k w_k q_k$.

many different trees

T_1 ————+—————

T_2 ———+———+———+———

T_3 ———+———+———

Classification trees: the bias is stuck

$$\text{bias} = \underbrace{\Pr_0(Y = y \mid X \in \text{bin})}_{\text{truth averaged over the bin}} - \underbrace{p_0(y \mid x)}_{\text{truth at } x}$$

Bins are **fixed** $\Rightarrow$ each bin blurs the same patch $\Rightarrow$ **bias can't shrink** (more data = tighter, not truer).

Fix: average **many trees** with *different* splits: $q = \sum_k w_k q_k$.

many different trees



weight w_k = how well tree k fits $\mathcal{D}_n$



predicts the real labels better $\Rightarrow$ bigger weight

Classification trees: the bias is stuck

$$\text{bias} = \underbrace{\Pr_0(Y = y \mid X \in \text{bin})}_{\text{truth averaged over the bin}} - \underbrace{p_0(y \mid x)}_{\text{truth at } x}$$

Bins are **fixed** $\Rightarrow$ each bin blurs the same patch $\Rightarrow$ **bias can't shrink** (more data = tighter, not truer).

Fix: average **many trees** with *different* splits: $q = \sum_k w_k q_k$.

many different trees



weight w_k = how well tree k fits $\mathcal{D}_n$



predicts the real labels better $\Rightarrow$ bigger weight

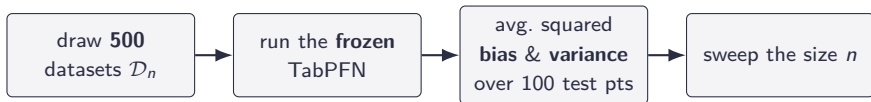
n grows $\Rightarrow$ best-fitting tree wins $\Rightarrow$ bias drops.

Experiment: the setup [Fig. 1]

The truth: $p_0(1 \mid x) = \frac{1}{2}(1 + \sin(\mathbf{1}^\top x)) \quad x \sim \mathcal{N}(\mathbf{0}, I_5).$

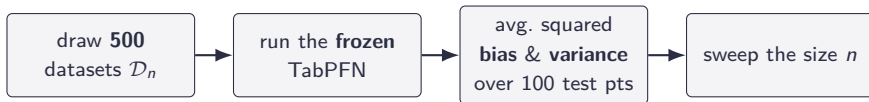
Experiment: the setup [Fig. 1]

The truth: $p_0(1 | x) = \frac{1}{2}(1 + \sin(\mathbf{1}^\top x))$ $x \sim \mathcal{N}(\mathbf{0}, I_5)$.



Experiment: the setup [Fig. 1]

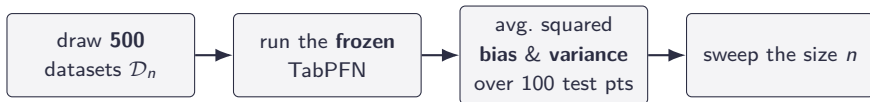
The truth: $p_0(1 | x) = \frac{1}{2}(1 + \sin(\mathbf{1}^\top x))$ $x \sim \mathcal{N}(\mathbf{0}, I_5)$.



Run *two* predictors head-to-head: **TabPFN** vs **Localized TabPFN** ($k_n = \min\{500, \lceil n^{4/(d+4)} \rceil\}$).

Experiment: the setup [Fig. 1]

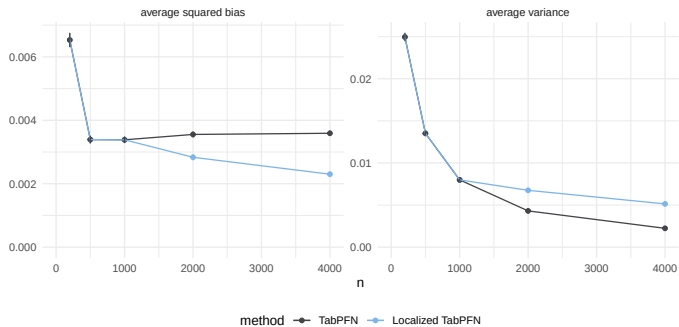
The truth: $p_0(1 | x) = \frac{1}{2}(1 + \sin(\mathbf{1}^\top x))$ $x \sim \mathcal{N}(\mathbf{0}, I_5)$.



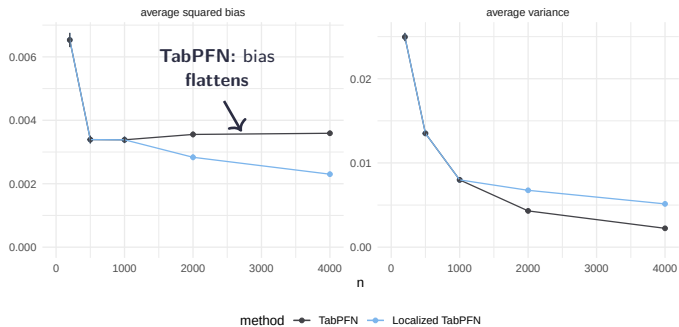
Run *two* predictors head-to-head: **TabPFN** vs **Localized TabPFN** ($k_n = \min\{500, \lceil n^{4/(d+4)} \rceil\}$).

Theory predicts: variance $\rightarrow 0$ at rate $1/n$; bias drops until $n \approx 1000$, then *stalls*.

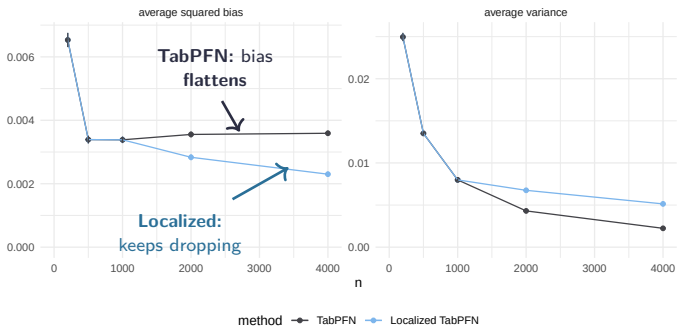
Experiment: the result [Fig. 1]



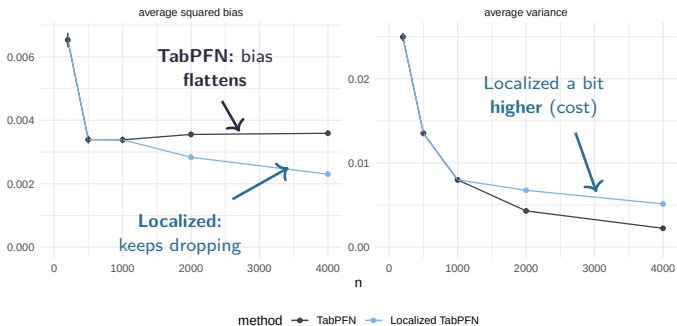
Experiment: the result [Fig. 1]



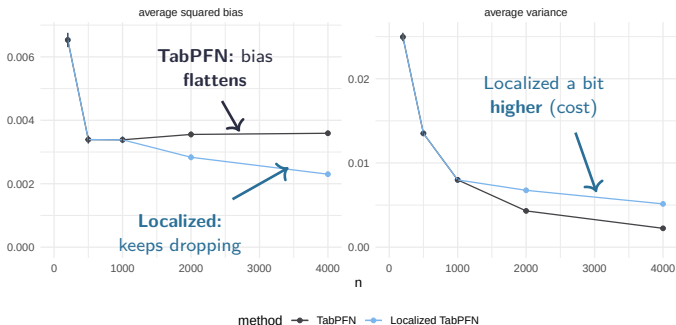
Experiment: the result [Fig. 1]



Experiment: the result [Fig. 1]



Experiment: the result [Fig. 1]



⇒ localizing breaks the bias floor, at a small variance cost.